DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this session

What we are learning in this session

- Introduction
- Data Types and Variables
- Operators

Let’s Start

Introduction

Introduction

What is C#?

- C# is pronounced "C-Sharp".
- It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.
- C# has roots from the C family, and the language is close to other popular languages like C++ and Java.
- The first version was released in year 2002. The latest version, C# 13, was released in November 2024.

C# is used for:

- Mobile applications, Desktop applications, Web applications, Web services, Web sites, Games, VR, Database applications, And much, much more!

Introduction

Why Use C#?

- It is one of the most popular programming language in the world.
- It is easy to learn and simple to use.
- It has a huge community support.
- C# is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.
- As C# is close to C, C++ and Java, it makes it easy for programmers to switch to C# or vice versa.

Variable and Data Type

What is a Variable?

- A **variable** is like a storage box where you can keep data. Each variable has:
  - **Name:** So you can refer to it (e.g., age, name).
  - **Data Type:** So the computer knows what kind of data it will store (e.g., a number, a letter, or a true/false value).
  - **Value:** The actual data you put into the box (e.g., 25 for age).

What is a Data Type?

- **Data type** tells the computer what kind of data a variable will store. In C#, you must specify this when you create a variable. It ensures the variable behaves correctly (e.g., numbers can be added, and strings can be combined).

Variable and Data Type

Allowed in Variable Names

- You can start the name with a letter(a-z, A-Z).
- You can use numbers in the name, but not as the first character.
- You can use underscores in the name, and it can even be the first character.
- C# is case-sensitive, so myVariable and MyVariable are different variables.

Not Allowed in Variable Names

- Variable names cannot contain spaces. Use camelCase or underscores instead.
- Variable names cannot begin with a number.
- You cannot use special characters like @, #, $, %, ^, &, *, etc.
- C# has certain reserved keywords (e.g., int, class, namespace, etc.) that you cannot use as variable names directly.
- Hyphens are not allowed in variable names. Use underscores instead.

Data Type

C# Data Type	Range	Size	.NET Type	Notes
byte	0 to 255	1 byte	System.Byte	For small positive numbers.
sbyte	-128 to 127	1 byte	System.SByte	For small signed integers.
short	-32,768 to 32,767	2 bytes	System.Int16	For small integers.
ushort	0 to 65,535	2 bytes	System.UInt16	For small positive integers.
int	-2,147,483,648 to 2,147,483,647	4 bytes	System.Int32	Most commonly used for whole numbers.
uint	0 to 4,294,967,295	4 bytes	System.UInt32	For large positive numbers.
long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 bytes	System.Int64	For very large integers.

Data Type

C# Data Type	Range	Size	.NET Type	Notes
ulong	0 to 18,446,744,073,709,551,615	8 bytes	System.UInt64	For very large positive numbers.
float	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	4 bytes	System.Single	For floating-point numbers. Use f suffix (e.g., 3.14f).
double	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	8 bytes	System.Double	More precise than float. Default for decimal numbers.
decimal	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$	16 bytes	System.Decimal	Best for financial or monetary calculations. Use m suffix (e.g., 12.99m).
char	Any single Unicode character	2 bytes	System.Char	Use single quotes (e.g., 'A').
bool	true or false	1 byte	System.Boolean	For logical operations.

Nullable Types

- **Nullable Types in C#**
  - In C#, nullable types allow variables of value types (like int, double, bool, etc.) to hold a special value called null.

- **Why Use Nullable Types?**
  - A database query may return null for an empty field.
  - A variable that hasn't been assigned a value yet might need to signify "no value."

- **How to Declare a Nullable Type**

```
int? age = null; // Nullable integer
```

```
Nullable<int> age = null; // Equivalent to int?
```

readonly and const

- **const**
  - A **constant** is a value that is known at **compile-time** and **never changes.**
  - It must be initialized **when declared.**
  - It is **implicitly static,** meaning it belongs to the type (class/struct) rather than any instance.

```
const data_type field_name = value;
```

- **readonly**
  - A readonly field can only be assigned once, either:
    - At the time of declaration.
    - Or, in the constructor (for instance fields).

```
readonly data_type field_name;
```

const
readonly
required

Operators

- **Arithmetic Operators**
  - Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%)

- **Why Use Arithmetic Types?**
  - Arithmetic operators used to perform mathematical operations on numbers.

- **Example**

```
Addition (+), Subtraction (-), Multiplication
(*), Division (/), Modulus (%)
Examples: a + b, a * b
```

Operators

- **Logical Operators**
  - AND (&&), OR (||), NOT (!)

- **Why Use Logical Types?**
  - Logical operators are used for checking conditions

- **Example**

```
Examples: a && b, !a
```

Operators

- **Assignment Operators**
  - AND (&&), OR (||), NOT (!)

- **Why Use Assignment Types?**
  - Basic Assignment (=): a = 5
- Compound Assignments: Add and Assign (+=), Subtract and Assign (-=), Multiply and Assign (*=), etc.

- **Example**

```
Examples: a += 5, b *= 2
```

Thank you